

Learning Thermal Properties and Temperature Models of Electric Motors with Neural Ordinary Differential Equations

Wilhelm Kirchgässner, Oliver Wallscheid, Joachim Böcker

Power Electronics and Electrical Drives, Paderborn University, Paderborn, Germany

E-mail: kirchgaessner@lea.upb.de

Abstract—The importance of thermal monitoring in electric traction drives gains more and more momentum as overload capabilities have to be utilized increasingly to stay competitive in the electric vehicle industry. Among real-time capable algorithms for the temperature estimation of in-series-unaccessible motor components, lumped-parameter thermal networks denote a popular choice but require extensive expert knowledge and motor data sheet information for their design. In this work, it is shown that with emerging techniques from the cross section of machine learning and physical modeling of heat transfer with differential equations, new chances for accurate real-time capable thermal modeling on a completely measurement-data-driven base arise. A data-efficient training scheme for thermal neural networks and their generalization, neural ordinary differential equations, is presented that converges to cross-validated estimation accuracies of less than 2 K² mean squared error, and below 6 K worst-case error at only 149 model parameters.

Keywords—thermal modeling, neural networks, permanent magnet synchronous motor, system identification

I. INTRODUCTION

Most traction drive applications in the automotive industry require a high power and energy density since electric vehicle (EV) mass harms driving range and room is limited. At the same time, acceleration performance metrics are important for both, marketing and customer satisfaction, such that automobile manufacturers find themselves more and more pushing the motor operating points into overload regions due to reduced safety margins in the incorporated machine material. High-power operation leads to high thermal stress, which can harm certain components, e.g., the copper winding insulation in the stator or the rotor permanent magnets (PMs). Thus, demanding traction trajectories without jeopardizing the motor functionality at less cost of materials are possible only when thermal conditions are precisely known for maximum material utilization. Hence, commercial competitiveness, combined with increasing pricing pressure, continues to drive the improvement of temperature estimation models in the automotive industry and the research domain. While classic approaches, like those based on heat transfer theory or on electrodynamic machine models, necessitate design by an expert [1], black-box models likewise have been proven to be accurate when sufficient recorded measurements are available. Successful examples are simple linear regression [2] and deep residual

learning [3]. However, data-efficient learning algorithms are also important in the automotive industry, as the collection of large data sets is resource-heavy and no matter of course. Moreover, despite their high accuracy, black-box models – being abstract and detached from physical phenomena – do not provide information about the thermal properties of an investigated system. In this work, the gap between both, expert-based modeling of heat transfer with ordinary differential equation solvers and data-driven machine learning (ML) algorithms is closed by utilizing neural ordinary differential equations [4]. To assist related research, all code is published on GitHub¹.

II. STATE OF THE ART

Different approaches on electric motor temperature estimation with their own advantages and shortcomings were extensively studied in the last decades, and a comprehensive overview is given in [5]. Excluding compute-heavy methods like computational fluid dynamics (CFD) or finite element analysis (FEA) from the following due to their lack of real-time capability [6], several other methods lend themselves for the thermal monitoring task by inheriting only a small model parameter size.

One specific domain is characterized by the estimation of temperature-sensitive electrical model parameters. This includes invasive methods: By means of signal injection and subsequent measurement of the impact on the stator winding resistance or the inductances with laboratory equipment, one can infer the thermal state of the permanent magnets [7], [8]. Alternatively, a fundamental wave flux observer has been shown capable of assessing the temperature of permanent magnets through their reversible demagnetization in [9]. However, common for both approaches is a prohibitive electric model parameter sensitivity, which might let estimation errors soar in case of inaccurate modeling. Another noteworthy approach denotes lumped-parameter thermal networks (LPTNs) based on heat transfer theory. They show strong predictive performance at small model orders and, thus, real-time capability [10]–[12]. One downside, however, is the necessity of the drive train specifications, e.g., machine geometry, cooling junction etc., and the design

¹<https://github.com/wkirgsn/thermal-nn>

by an expert of the domain, making each LPTN device-specific not only in its parameters but also in its topology [13]–[15].

Recently, machine learning models were introduced [3], [16], [17], which rely entirely on measured data. In particular, the most accurate models among the field of supervised learning are linear regression and neural networks (NNs). For both, the adherence of differently spanned versions of the exponentially weighted moving average and likewise averaged moving standard deviation of each signal to the feature input matrix is essential for the solid estimation of the component temperatures. However, even with more efficient recursive calculations of these moving averages [18], the addition of such input features hurts real-time performance. Moreover, the choice of filter strengths is nontrivial as it requires experimental trial and error since no automatic optimization exists for feature engineering apart from heuristics like particle swarm optimization [19] or Bayesian optimization [20]. It is advantageous though that the contemplated sensors are available in series production vehicles' electronic control units, making additional measurement equipment redundant. The estimation performance is surprisingly competent considering the detachment of any physically motivated modeling, yet they depend on large data sets recorded on test benches driving diverse load profiles.

Large data sets might not exist in every industry, but some recordings are usually readily available due to prototypical research and development. The crucial point then is to have a learning model that can make the most of given small data sets. An increased data-efficiency can be achieved by introducing, again, the principles of heat transfer into the ML model as shown in [21] or with thermal neural networks (TNNs) in [22]. Although the space of functions that can be approximated becomes limited through incorporation of differential equations, learning can be accelerated and made more consistently successful when the data loosely corresponds to this model. The TNN brings a lot of advantages over the preceding thermal modeling techniques, e.g., through exhibiting physically interpretable states, initial conditions can be set; end-to-end differentiability; no expert knowledge requirement; and small model size with high accuracy enabling real-time usage, among others. However, it relies on the explicit Euler forward time discretization [23], and might suffer from the exploding and vanishing gradient problem [24].

III. THERMAL MODELING WITH NEURAL ORDINARY DIFFERENTIAL EQUATIONS

In a different domain of research, more and more general-purpose function approximating models are proposed that incorporate differential equations of different properties. Noteworthy mentions are neural ordinary differential equations (NODEs) [4], their generalization to universal differential equations [25], physics-informed neural networks [26] and derivations from these [27]–[29].

Particularly, NODEs represent initial value problems (IVPs) of the form

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (1)$$

with $\mathbf{x} \in \mathbb{R}^m$ denoting the system state at time $t \in [t_0, t_T]$, $\mathbf{u} \in \mathbb{R}^{\nu}$ the system input, and $\mathbf{f}_{\boldsymbol{\theta}} : \mathbb{R}^{m+\nu+1} \rightarrow \mathbb{R}^m$ being a neural network function parameterized with $\boldsymbol{\theta}$. A differentiable ODE solver replaces a hidden layer, which is potentially repeatedly executed to implicitly learn the latent function $\mathbf{f}_{\boldsymbol{\theta}}$. In this work, the function approximator $\mathbf{f}_{\boldsymbol{\theta}}$ is not a single NN but rather a composite of several feed-forward NNs that predict certain thermal properties, similar to a TNN but without the limit on the Euler forward discretization and a different gradient calculation routine.

For one-dimensional heat transfer the ordinary differential equation (ODE) of an m -th order, fully-interconnected LPTN is given for each node i as:

$$C_i(\boldsymbol{\zeta}(t)) \frac{d\vartheta_i}{dt} = P_i(\boldsymbol{\zeta}(t)) + \sum_{j \in \mathcal{M} \setminus i} \frac{\vartheta_j - \vartheta_i}{R_{i,j}(\boldsymbol{\zeta}(t))} + \sum_{j=1}^n \frac{\tilde{\vartheta}_j - \vartheta_i}{R_{i,j}(\boldsymbol{\zeta}(t))}, \quad (2)$$

where $\mathbf{C} \in \mathbb{R}^m$ denotes the thermal capacitances, $\mathbf{P} \in \mathbb{R}^m$ the power losses, $R_{i,j} \in \mathbb{R}$ the thermal resistance between the temperatures at node i and j , and $\boldsymbol{\vartheta} \in \mathbb{R}^m$ the actual temperature at time t with $\mathcal{M} = \{1, 2, \dots, m\}$, while ancillary temperatures such as ambient or coolant temperatures are denoted $\tilde{\boldsymbol{\vartheta}}$. Note that, especially for low-order models with a low spatial discretization, the thermal parameters are usually operation-point-varying rather than being constant, which is indicated by the thermal parameters' dependence on an unknown scheduling vector $\boldsymbol{\zeta}(t)$.

Combining the idea of (1) with (2), it is proposed that

$$\frac{d\vartheta_i}{dt} = \kappa_i \left(\mathbf{p}_i(\boldsymbol{\vartheta}, \mathbf{u}, t; \boldsymbol{\theta}_p) + \sum_{j \in \mathcal{M} \setminus i} (\vartheta_j - \vartheta_i) \mathbf{g}_{i,j}(\boldsymbol{\vartheta}, \mathbf{u}, t; \boldsymbol{\theta}_g) + \sum_{j=1}^n (\tilde{\vartheta}_j - \vartheta_i) \mathbf{g}_{i,j}(\boldsymbol{\vartheta}, \mathbf{u}, t; \boldsymbol{\theta}_g) \right), \quad (3)$$

with $\kappa_i \in \mathbb{R}$ being the learnable scalar inverse thermal capacitance at node i , and $\mathbf{p} : \mathbb{R}^{m+n+\nu} \rightarrow \mathbb{R}^m$ denoting an arbitrary feed-forward NN parameterized by $\boldsymbol{\theta}_p$ that outputs the power loss and is informed by estimated states and system inputs, while

$$\mathbf{g} : \mathbb{R}^{m+n+o} \rightarrow \mathbb{R}^{(m+n)(m+n-1)/2}$$

denotes likewise an arbitrary NN parameterized by $\boldsymbol{\theta}_g$ with the same input information but outputs the thermal conductance between each possible node pair. The hidden temperatures $\boldsymbol{\vartheta}$ denote the system state from here on and replace $\mathbf{x}$. The solution of the IVP as stated by initial temperatures and (3) can be obtained by an arbitrary ODE solver, making the NODE approach more generally applicable than TNNs. Specifically, arbitrary points in

time can be solved for that would not be tractable by common time-discrete models.

A. Adjoint State Method

In contrast to a TNN that comes with a trivial Euler forward discretization and is trained by truncated back-propagation through time (TBPTT) [30], choosing more sophisticated solvers poses the question for efficiently solving the optimization problem when constrained by an ODE and given hundreds of model parameters. As shown in [4], this can be done with the adjoint state method: Having solved the IVP (3) with a certain solver for a certain initial value ϑ_0 , a corresponding system input trajectory u , and given model parameters θ , a solution trajectory $\hat{\vartheta}(t)$ is returned which is evaluated under a cost $J(\hat{\vartheta}, \vartheta)$ (usually the mean squared error (MSE)), for which sensitivities $dJ/d\theta$ need to be obtained – note the hat-notation for estimates. The adjoint state method provides a way to obtain these with linear computational complexity with respect to the amount of model parameters by solving a terminal value problem (TVP) after solving the IVP in the context of an ODE-equality-constrained optimization problem [31]. More specifically, the adjoint state is defined as

$$a(t) = \frac{\partial J}{\partial \hat{\vartheta}(t)}, \quad (4)$$

whose dynamics are given by (proof in appendix of [4])

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(\hat{\vartheta}(t), u, t, \theta)}{\partial \hat{\vartheta}(t)}. \quad (5)$$

The initial value $a(t_0)$ can be retrieved by solving the TVP given by (5) and $a(t_T)$, again with an ODE solver, which does not need to be the same as for the forward pass. Having obtained trajectories $\hat{\vartheta}$ and a along $t \in [t_0, t_T]$, the gradient calculation of the loss with respect to model parameters will be enabled through

$$\frac{\partial J}{\partial \theta} = - \int_{t_0}^{t_T} a(t)^T \frac{\partial f(\hat{\vartheta}(t), u, t, \theta)}{\partial \theta} dt. \quad (6)$$

As explained in [4], all vector-Jacobian products are computationally tractable through automatic differentiation and all integrals can be obtained by a single backward-in-time ODE call on an augmented state vector.

B. Tensor Feeding Setup

Usually, data sets containing temperature trajectories consist of multiple measurement profiles of varying lengths. The obvious way of feeding these profiles into a model would be sequentially, but it is more common (and efficient) to apply (mini-)batch training instead. This training scheme implies to propagate all or a subset of measurement profiles through an equal amount of model copies and the calculation of the gradients in parallel through matrix multiplications, which is followed by a weight update on all model copies with the average gradient across all profiles and points in time within a certain TBPTT length. In the common case of measurement profiles with different lengths, post-padding with

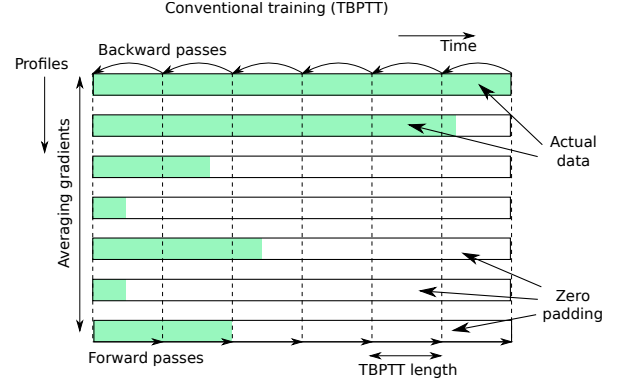


Fig. 1. The conventional training scheme [22]: Gradients will be accumulated from start to end for all measurement profiles in parallel and their masked average after certain time periods defines the subsequent model weight updates.

zeros and the multiplication of the loss per predicted observation with zero likewise is suggested to avoid ill-informed gradients (see Fig. 1). During training, diverged estimates will be taken over as initial state values for the next TBPTT block. While this is efficient for TNN training with TBPTT as shown in [22], for a NODE this is less favorable due to the zero-padded terminal values that denote the starting point of the adjoint state solution. Moreover, even for a TNN this training scheme might suffer from far-propagated errors that stem from unoptimized model weights in the beginning of an epoch, as well as from highly stochastic updates in the last half of an epoch, where only few profiles provide information.

As a remedy, a training scheme with subsequences is proposed which affects the tensor feeding setup to the model. All measurement profiles are broken down into subsequences of a certain length (here, one hour), which denote independent new profiles in which most will have a defined terminal value (see Fig. 2). In each epoch, random subsets of subsequences will be assigned to multiple mini-batches (MBs). The average of the gradients over one MB denote one model weight update. Additionally, when training TNNs with this scheme, a TBPTT length smaller than the subsequence length can be chosen for multiple weight updates within a MB. For such a tensor feeding setup, the subsequence length and the amount of subsequences per MB represent additional hyperparameters that can be optimized.

C. Model Topology

The model architecture is chosen to be one of the smallest as found in [22] during a hyperparameter optimization, and is not further adapted, except for the activation function in the intermediate hidden layers. There, the hyperbolic tangent (TanH) is employed instead of the rectified linear unit, due to the smoother gradient, which is to be preferred for NODEs. Apart from that, the topology for the remainder of this work can be described as follows (see Fig. 3), which consists of 149 model parameters:

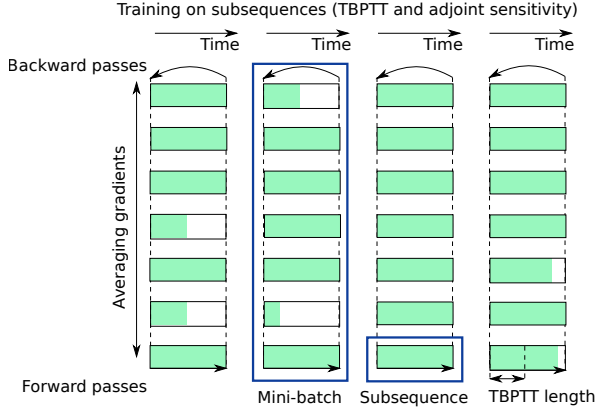


Fig. 2. The MB training scheme on fixed-length subsequences: For NODEs, gradients will be retrieved by backwards-solving an ODE from end to start, parallelized over a certain amount of subsequences that define a MB. For a TNN, multiple consecutive weight updates are possible while propagating through subsequences in a MB.

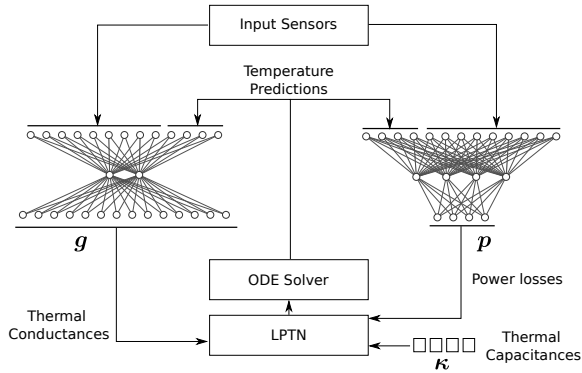


Fig. 3. The optimized topology according to [22] with 149 model parameters. The thermal capacitances denote mere trainable constants. Biases are not shown.

1) *Thermal conductances*: On the input sensor features and the last predicted temperatures follows a two-unit hidden layer, the TanH activation, an output layer with as many nodes as there are temperature pairs with unknown heat paths (here, 14), and the exponential linear unit activation added by one, in order to have positive output values.

2) *Power losses*: On the same input constellation as for the previous sub-net follow two four-unit layers, the former with the TanH and the latter with a sigmoid activation function. These outputs denote the normalized total losses generated in each component.

3) *Thermal capacitances*: These thermal properties are derived from learned scalar values $\theta_{c,i}$ where $i \in \mathcal{M}$, one for each component. In particular, the exponential activation function is applied on the learned parameters: $\kappa = \exp(\theta_c)$, with $\kappa \in \mathbb{R}^m$ being the inverse thermal capacitances. This setup helps to keep training numerically stable and to accelerate the gradient descent optimization.

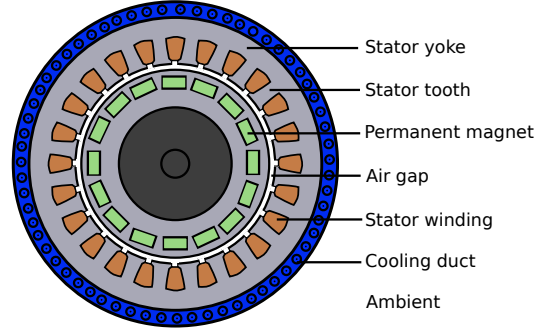


Fig. 4. Sketch of the PMSM cross section (cf. [22]).



Fig. 5. Test bench with comparable PMSM in the lab (cf. [3]).

IV. EXPERIMENTAL RESULTS

For all modeling experiments we use TorchDiffEq [4], Pytorch [32], and the open-source² permanent magnet synchronous motor (PMSM) data set from [3]. A schematic cross section of the motor under investigation is seen in Fig. 4 and a picture of the test bench in the lab is shown in Fig. 5. The most important characteristics of the test bench are compiled in Tab. I. In this work, the most interesting temperatures to monitor are those for the following components: the stator winding ϑ_{sw} , stator tooth ϑ_{st} , stator yoke ϑ_{sy} , and especially the permanent magnets ϑ_{pm} . All temperatures are measured with type K thermocouples, while the rotor temperature information is transmitted wirelessly over a telemetry unit. All signals are sampled with 2 Hz. The loss function throughout all experiments is the MSE between measured and predicted temperatures, averaged over the four target components.

The data set is split into a training set (65 profiles, 171 h), a validation set (one profile, 4.6 h), and a test set (three profiles, 8.9 h). The validation set is used to indicate generalization performance and possibly early stopping during training after each epoch, while the test set is evaluated after training for the true generalization capability. Only profiles in the training set will be broken

²<https://www.kaggle.com/wkirsnsn/electric-motor-temperature>

TABLE I. TEST BENCH PARAMETERS

Motors	device under test	load motor
Type	PMSM	induction machine
Power rating (nom./max.)	22/52kW	160/210kW
Voltage rating	177 V	380 V
Current rating (nom./max.)	110/283A	293/450A
Torque rating (nom./max.)	110/250 Nm	380/510 Nm
Pole pair number	8	1
Cooling	water-glycol	forced air
Thermocouples	type K - class 1	-
Inverter	3 × SKiiP 1242GB120-4DW	
Topology	voltage source inverter	
	2-level, IGBT	
Inverter interlocking time	3.3 μs	
Controller hardware	dSPACE	
Processor board	DS1006MC, 4 cores, 2.8 GHz	

down into subsequences for the optimized tensor feeding setup.

All models are fed with the following nine input sensor signals: currents and voltages in d/q coordinates as well as their ℓ_2 -norm, motor speed, and the ambient and coolant temperatures. All input and target temperatures are normalized with division by 100 °C, whereas all other input sensors are divided by their maximum absolute quantity occurring in the data set.

A. Generalization Capability

The effectiveness of the training algorithm with subsequences becomes evident in Fig. 6. Models trained with the MB tensor feeding setup, i.e., b) - e), achieve more consistent convergence to better optima than the classic TNN setup from [22]. Moreover, it can be seen that NODE variants can further scale their accuracy with more training epochs in contrast to TNNs. On the other hand, the better convergence properties have to be bought with longer training times since CPU utilization is lessened through MB training that forces fewer profiles to be parallelized over. However, this is a property of the data set and would look different for other amounts of measurement profiles, and is also adjustable to some extent by the hyperparameters subsequence length and amount of MBs. The increased time cost for NODE training, nonetheless, is independent of the data set since gradients are obtained by solving a TVP.

The best seed's performance for a NODE after training is displayed in Fig. 7, which highlights a desirable cross-validated average MSE score under 2 K and, at the same time, a low worst-case error of under 5.7 K. All seeds for the best NODE variant are compared with those of a TNN with conventional and MB training methods in Fig. 8, where the superiority of the subsequenced tensor feeding setup becomes evident again. The average MSE across the four target components is slightly better than for a TNN, which emphasizes the adjoint state method's advantage during gradient descent optimization of a state-space structure. Total accuracy is better than current state-of-the-art monitoring algorithms [17], even than those that are supplied with extensive look-up tables (LUTs) of measured power loss maps (e.g., LPTNs), while the amount of model parameters is below 150. At the same time, no

TABLE II. STATE-OF-THE-ART THERMAL MODELING PERFORMANCE METRICS AS EVALUATED ON THE TEST SET

	MSE (K ²)	ℓ_∞ (K)	LUT entries	Feature entries	Model size
MLP [17]	5.58	14.29	0	81	1444
OLS [17]	4.47	9.85	0	81	328
CNN [3]	4.43	15.54	0	81	4916
LPTN [10]	3.64	7.37	12810	4	34
TNN (conventional) [22]	1.64	6.58	0	9	149
TNN (this work)	1.08	5.45	0	9	149
NODE	1.24	5.68	0	9	149

feature engineering in form of costly moving averages has to be designed and, thus, also does not computationally burden embedded system implementations. The accuracy comparison between the models is compiled in Tab. II. All numbers show the performance of the corresponding best random seed. Among the MSE and the worst-case error ℓ_∞ , the number of LUT entries, input feature entries, and the model sizes are highlighted. It becomes evident that a TNN has achieved the highest performance with the optimized training scheme, yet with a look on Fig. 8, the NODE algorithm converges to a better optimum more consistently.

The possibility to choose other solvers than the explicit Euler was investigated for the constellation of Fig. 6 e), where a Runge-Kutta of 4th order (RK4) was employed for the backward solving routine. However, the results were nearly identical, so the chances of having a choice for the solver remains theoretical for this work's data set, but might aid for time series data that is less frequently or irregularly sampled.

B. Learned Thermal Properties

Similar to LPTNs and TNNs, the neural ODE also features interpretable model states, that can help gaining insights about the thermal properties of the system. In particular, the thermal conductances and power losses are expected to vary with the operation, while the thermal capacitances are constants by definition. The variation of the thermal conductances and power losses for the best NODE random seed when propagated through the whole data set are visualized in Fig. 9 and Fig. 10, respectively.

It can be deduced from the thermal conductance distribution that the heat paths between stator yoke and teeth (SY-ST); the stator yoke and coolant (SY-C); and between stator teeth and winding (ST-SW) are consistently significant. In contrast, the heat paths for the pairs permanent magnet to stator winding (PM-SW) and stator winding to ambient temperature (SW-A) are mostly of low thermal conductance, and might be neglected for subsequent experimental iterations. This would lead to sparser models [22]. The highest variances occur for pairs PM-SY, PM-ST, and PM-C, which are most of those paths leading to the PM. This signals a multi-medium path where heat transfer not only through conduction, but also convection and radiation are involved. That finding aligns with the actual geometry and constellation of the components within the motor, which apparently has been successfully identified by the model.

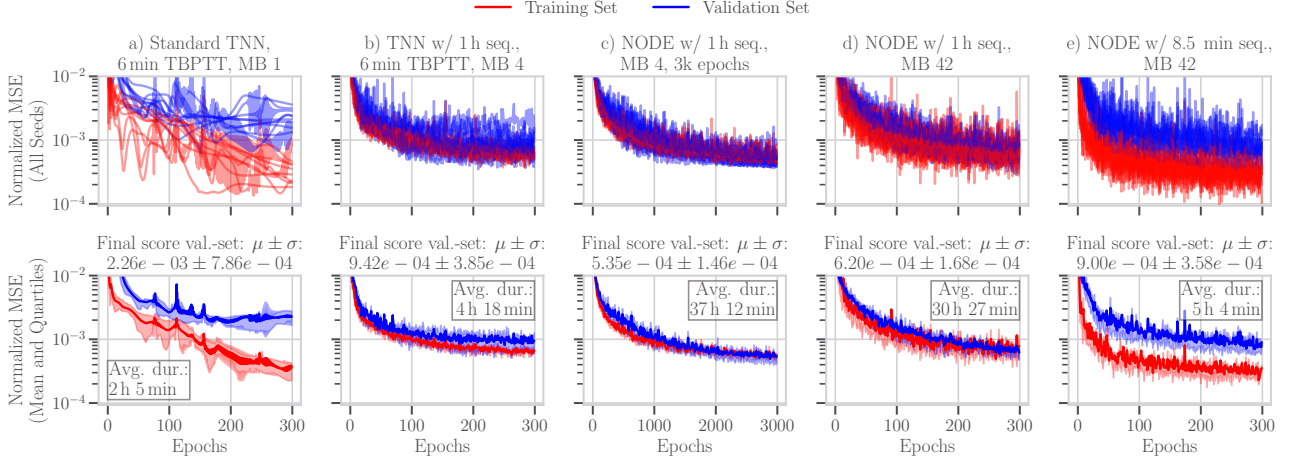


Fig. 6. Training and validation set trends over epochs for different models and mini-batches (MB) per epoch. All variants experienced the same amount of weight updates. First row shows all 10 random seeds and second the mean and 25%/75% quartile bands across all seeds. a) A standard TNN with the conventional tensor feeding setup overfitting as of 200 epochs. b) A TNN with the subsequenced setup performs better with no evident overfitting. c)-e) NODE learning with certain hyperparameters scales better with the number of epochs than TNN learning.

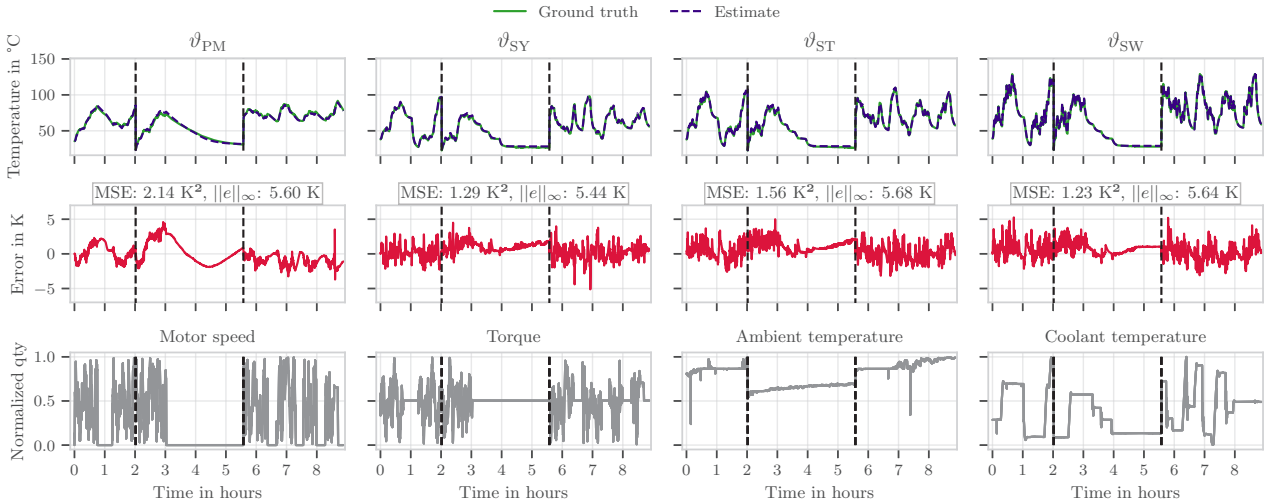


Fig. 7. Best NODE seed performance on the test set (three independent profiles, separated by vertical dashed lines). The bottom row shows four selected sensor signals. Note that torque was not part of the input feature set.

From Fig. 10 it becomes obvious that power losses in the SW tend to be highest, followed by the SY, PM, and eventually ST. Such finding might aid in subsequent motor designs when combined with simulation data.

In Fig. 11, the thermal capacitances across all ten random seeds of the best NODE variant are illustrated, in contrast to the previous two thermal properties that showcased only the best random seed. The shrinking variance with lower thermal capacitance can be explained by the exponential tuning of the corresponding model weights. A consistent order of PM-SY-ST-SW, from highest to lowest, is evident, but at the same time, the random convergence of gradient descent optimization into different local optima becomes clear as, e.g., the PM's thermal capacitance might vary significantly depending on the training outcome. Likewise, some variation in the thermal conductance and power loss distribution across random seeds can be expected.

Although the thermal properties according to the investigated models are given in normalized quantities, actual physical magnitudes can be obtained if either of the power losses, the thermal conductances, or the thermal capacitances is identified at the test bench, e.g., through power loss measurement with auxiliary equipment.

C. Model Stability

Moreover, the passiveness of the system due to the positiveness constraint on the thermal parameters (as enforced through activation functions on the output of $\mathbf{g}$ and $\mathbf{p}$) enables a stable estimation, that is not harmed by detuned initial conditions. The convergence of estimated temperatures to the real ground truth given a skewed initial guess is illustrated in Fig. 12. It becomes clear that initial temperature deviations of up to 50°C are mostly recovered within 10 min for stator temperatures, and 30 min for the permanent magnet temperatures. The

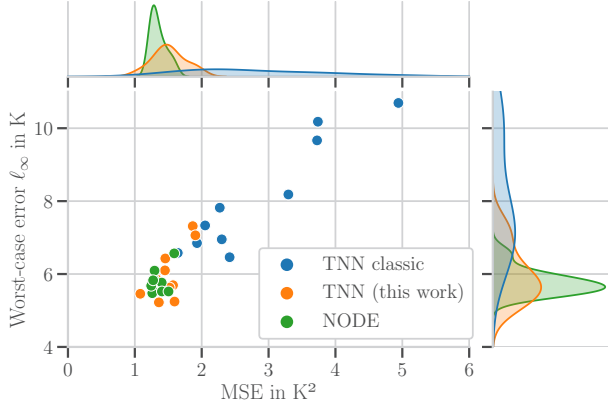


Fig. 8. All random seed generalization capabilities in a MSE- ℓ_∞ -plane per modeling approach. The further to the bottom-left corner the better.

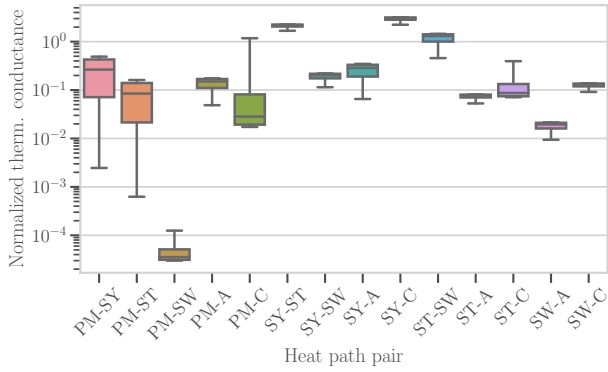


Fig. 9. Thermal conductances distribution for each component pair as estimated by the best NODE seed across the whole data set

recovery times could be further reduced by incorporating the TNN/NODE into a closed-loop estimator like a Kalman filter.

V. CONCLUSION

In this work, two main findings are demonstrated: first, the effectiveness of training data-driven thermal models on subsequenced data in a mini-batch fashion, which shows better convergence properties consistently. Second, the high estimation accuracy at small model sizes

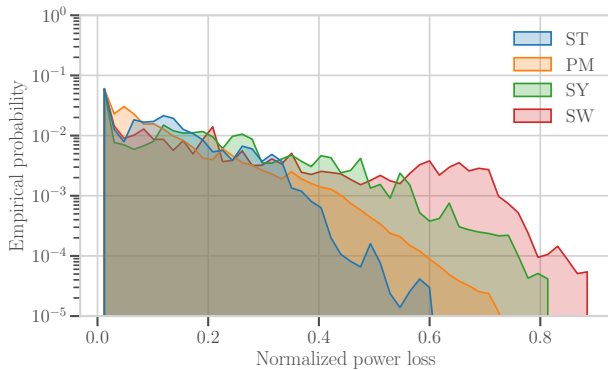


Fig. 10. Power loss distribution for each component as estimated by the best NODE seed across the whole data set

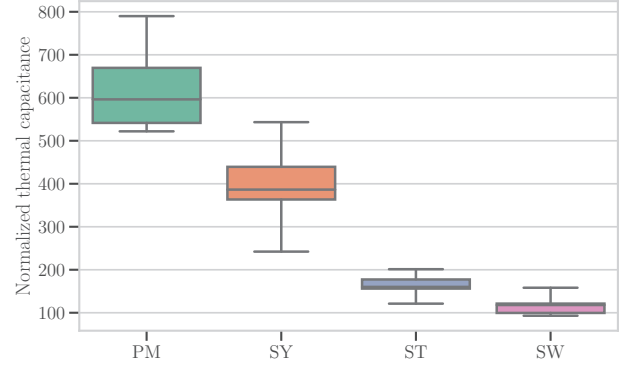


Fig. 11. Thermal capacitances distribution for each component as estimated by all ten NODE seeds (independent of motor operation)

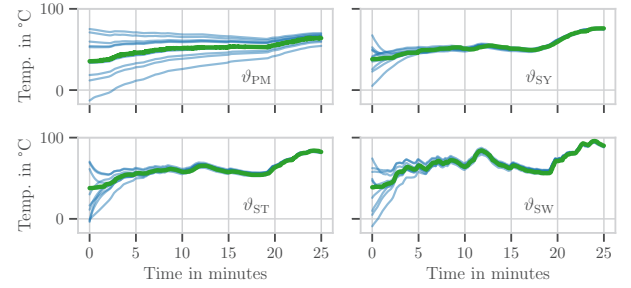


Fig. 12. Detuned initial conditions do not harm stability. The first 25 minutes of the first test set profile are illustrated with the ground truth in green and 10 random initial states in blue as estimated by the best NODE model.

of NODEs that provide interpretable model states. As a generalization of TNNs, a NODE can be potentially employed for irregularly or low-frequently sampled data sets. Moreover, the gradient calculation does not suffer from the exploding and vanishing gradient problem due to the adjoint state method, which expresses itself in a better performance scaling with more training epochs. However, since gradient calculation requires solving a TVP, training time per epoch is considerably longer than for a TNN.

The presented models show unrivaled estimation performance at low computational loads due to the small model sizes, such that real-time capabilities on embedded systems for thermal monitoring are possible.

Future research might focus on optimizing the newly identified hyperparameters and application to neighboring domains of research, such as lumped-parameter thermal modeling of power electronic modules, batteries and fuel cells, or other motor types. Additionally, optimal design of experiments through system excitation planning remains an open research question.

VI. ACKNOWLEDGMENTS

This work is funded by the Deutsche Forschungsgemeinschaft (German Research Foundation, DFG) under grant BO 2535/15-1. Computing time was provided by the Paderborn Center for Parallel Computing (PC²).

REFERENCES

- [1] O. Wallscheid, T. Huber, W. Peters, and J. Böcker, "Real-Time Capable Methods to Determine the Magnet Temperature of Permanent Magnet Synchronous Motors - A Review," in *40th Annual Conference of the IEEE Industrial Electronics Society*, 2014, pp. 811–818.
- [2] W. Kirchgässner, O. Wallscheid, and J. Böcker, "Empirical Evaluation of Exponentially Weighted Moving Averages for Simple Linear Thermal Modeling of Permanent Magnet Synchronous Machines," in *Proceedings of the 28th International Symposium on Industrial Electronics*, 2019, pp. 318–323.
- [3] —, "Estimating Electric Motor Temperatures with Deep Residual Machine Learning," *IEEE Transactions on Power Electronics*, vol. 36, no. 7, pp. 7480–7488, 2020.
- [4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural Ordinary Differential Equations," in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018, pp. 6571–6583.
- [5] O. Wallscheid, "Thermal Monitoring of Electric Motors: State-of-the-Art Review and Future Challenges," *IEEE Open Journal of Industry Applications*, vol. 2, pp. 204–223, 2021.
- [6] A. Boglietti, A. Cavagnino, D. Staton, M. Shanel, M. Mueller, and C. Mejuto, "Evolution and Modern Approaches for Thermal Analysis of Electrical Machines," *IEEE Transactions on Industrial Electronics*, vol. 56, no. 3, pp. 871–882, 2009.
- [7] D. D. Reigosa, F. Briz, P. Garcia, J. M. Guerrero, and M. W. Degner, "Magnet Temperature Estimation in Surface PM Machines Using High-Frequency Signal Injection," *IEEE Transactions on Industry Applications*, vol. 46, no. 4, pp. 1468–1475, 2010.
- [8] D. Fernandez, M. Martinez, D. Reigosa, A. B. Diez, J. M. Guerrero, and F. Briz, "Comparative Analysis of Magnet Thermal and Magnetization State Monitoring in PMSMs Based on High Frequency Signal Injection," *IEEE Transactions on Industry Applications*, 2019.
- [9] O. Wallscheid, A. Specht, and J. Böcker, "Observing the Permanent-Magnet Temperature of Synchronous Motors Based on Electrical Fundamental Wave Model Quantities," *IEEE Transactions on Industrial Electronics*, vol. 64, no. 5, pp. 3921–3929, 2017.
- [10] O. Wallscheid and J. Böcker, "Global Identification of a Low-Order Lumped-Parameter Thermal Network for Permanent Magnet Synchronous Motors," *IEEE Transactions on Energy Conversion*, vol. 31, no. 1, pp. 354–365, 2016.
- [11] E. Gedlu, O. Wallscheid, and J. Böcker, "Permanent Magnet Synchronous Machine Temperature Estimation using Low-Order Lumped-Parameter Thermal Network with Extended Iron Loss Model," in *The 10th International Conference on Power Electronics, Machines and Drives (PEMD 2020)*, 2020, pp. 937–942.
- [12] D. Liang, Z. Q. Zhu, Y. Zhang, J. Feng, S. Guo, Y. Li, J. Wu, and A. Zhao, "A Hybrid Lumped-Parameter and Two-Dimensional Analytical Thermal Model for Electrical Machines," *IEEE Transactions on Industry Applications*, vol. 57, no. 1, pp. 246–258, 2021.
- [13] F. Qi, M. Schenk, and R. W. De Doncker, "Discussing Details of Lumped Parameter Thermal Modeling in Electrical Machines," in *7th IET International Conference on Power Electronics, Machines and Drives (PEMD 2014)*, 2014, pp. 1–6.
- [14] D. Wöckinger, G. Bramerdorfer, S. Drexler, S. Vaschetto, A. Cavagnino, A. Tenconi, W. Amrhein, and F. Jeske, "Measurement-Based Optimization of Thermal Networks for Temperature Monitoring of Outer Rotor PM Machines," in *2020 IEEE Energy Conversion Congress and Exposition (ECCE)*, 2020, pp. 4261–4268.
- [15] D. Wöckinger, G. Bramerdorfer, S. Vaschetto, A. Cavagnino, A. Tenconi, W. Amrhein, and F. Jeskey, "Approaches for Improving Lumped Parameter Thermal Networks for Outer Rotor SPM Machines," in *2021 IEEE Energy Conversion Congress and Exposition (ECCE)*, 2021, pp. 3821–3828.
- [16] J. Lee and J. Ha, "Temperature Estimation of PMSM Using a Difference-Estimating Feedforward Neural Network," *IEEE Access*, vol. 8, pp. 130 855–130 865, 2020.
- [17] W. Kirchgässner, O. Wallscheid, and J. Böcker, "Data-Driven Permanent Magnet Temperature Estimation in Synchronous Motors with Supervised Machine Learning: A Benchmark," *IEEE Transactions on Energy Conversion*, vol. 36, no. 3, pp. 2059–2067, 2021.
- [18] T. Finch, "Incremental Calculation of Weighted Mean and Variance," *University of Cambridge*, vol. 4, no. 11-5, pp. 41–42, 2009.
- [19] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. IEEE, 1995, pp. 1942–1948.
- [20] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the Human out of the Loop: A Review of Bayesian Optimization," in *Proceedings of the IEEE*, vol. 104, no. 1, 2016, pp. 148–175.
- [21] E. G. Gedlu, O. Wallscheid, and J. Böcker, "Temperature Estimation of Electric Machines using a Hybrid Model of Feed-Forward Neural and Low-Order Lumped-Parameter Thermal Networks," in *2021 IEEE International Electric Machines Drives Conference (IEMDC)*, 2021, pp. 1–8.
- [22] W. Kirchgässner, O. Wallscheid, and J. Böcker, "Thermal Neural Networks: Lumped-Parameter Thermal Modeling With State-Space Machine Learning," *ArXiv e-prints: 2103.16323[cs.LG]*, 2021.
- [23] J. C. Butcher, *Numerical methods for ordinary differential equations*. New York: John Wiley & Sons, 2016.
- [24] R. Pascanu, T. Mikolov, and Y. Bengio, "Understanding the Exploding Gradient Problem," *Proceedings of The 30th International Conference on Machine Learning*, 2012.
- [25] C. Rackauckas, Y. Ma, J. Martensen, C. Warner, K. Zubov, R. Supekar, D. Skinner, and A. Ramadhan, "Universal Differential Equations for Scientific Machine Learning," *ArXiv e-prints: 2001.04385[cs.LG]*, 2020.
- [26] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [27] G. S. Misyris, A. Venzke, and S. Chatzivasileiadis, "Physics-Informed Neural Networks for Power Systems," 2019.
- [28] S. Massaroli, M. Poli, J. Park, A. Yamashita, and H. Asama, "Dissecting Neural ODEs," *ArXiv e-prints: 2002.08071[cs.LG]*, 2020.
- [29] S. Bai, J. Z. Kolter, and V. Koltun, "Deep Equilibrium Models," *ArXiv e-prints: 1909.01377[cs.LG]*, 2019.
- [30] R. J. Williams and J. Peng, "An Efficient Gradient-Based Algorithm for On-line Training of Recurrent Network Trajectories," *Neural computation*, vol. 2, no. 4, pp. 490–501, 1990.
- [31] B. Sengupta, K. J. Friston, and W. D. Penny, "Efficient Gradient Computation for Dynamical Models," *NeuroImage*, vol. 98, pp. 521–527, 2014.
- [32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshine, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035.